

# Delegação Capacitária com Profundidade Limitada

Um modelo de controle de acesso distribuído sobre CASL,  
com grafo de delegação, intensidade de propagação,  
e *enforcement* por *Row-Level Security*

Patria Technology — patria-erp

Maio de 2026

## Resumo

Modelamos o controle de acesso do **patria-erp** como *delegação capacitária*: autoridades intrínsecas (o operador da plataforma, que tudo vê; e cada *tenant*, com controle total dos próprios recursos) emitem *capacidades* que fluem por um **grafo de delegação** — não uma árvore: um concedente pode autorizar *diretamente* qualquer principal. Duas teses estruturam o modelo: (i) **desacoplar o *tenant***, que deixa de ser eixo especial e vira apenas *uma condição* dentro de uma capacidade; e (ii) dotar cada concessão de uma **intensidade**  $\delta$  — a *profundidade de propagação* — que limita quantas re-delegações ela admite ( $\delta = 0$ : o destinatário usa mas não compartilha;  $\delta = k$ : re-delegável por mais  $k$  saltos, decrementando;  $\delta = \infty$ : irrestrito). Mostramos que a delegação é uma *atenuação* (condição  $\subseteq$  e  $\delta$  decrementado), o que garante não-escalada de privilégio por construção, e que o *enforcement* se reduz a empurrar a *união* das condições detidas para o **where** das consultas (*pushdown*), realizado por uma extensão do Prisma Client. Um ponto operacional central:  $\delta$  é verificado no **ato de delegar**, não na leitura. Por fim, modelamos o **ciclo de vida** de cada concessão como uma máquina de estados (*active/suspended/revoked/expired*): a revogação *cascadeia* pela cadeia de delegação e a expiração se propaga sozinha pelo invariante de atenuação ( $\tau_{\text{filho}} \leq \tau_{\text{pai}}$ ). Por baixo de tudo, caracterizamos o sistema como um **fluxo monótono sobre um reticulado booleano** — semianel idempotente  $(2^{\mathcal{R}}, \cup, \cap)$  para a autorização, operador de atenuação deflacionário  $T \preceq \text{id}$  para a delegação — do que os teoremas (não-escalada, propagação limitada, fail-closed) seguem como corolários. A profundidade  $\delta$ , por sua vez, é a iteração do operador de *propagação*  $P \succeq \text{id}$  (dual da atenuação) sobre os conjuntos de principais, com  $\delta = \infty$  no seu *menor* ponto fixo  $\mu P$  (fecho transitivo). O Apêndice A traz o plano de implementação.

## Sumário

|     |                                                               |   |
|-----|---------------------------------------------------------------|---|
| 1   | Introdução e motivação                                        | 2 |
| 2   | Preliminares                                                  | 2 |
| 3   | Autorização de um principal                                   | 3 |
| 4   | O grafo de delegação e a atenuação                            | 3 |
| 4.1 | A intensidade $\delta$ : profundidade de propagação . . . . . | 4 |
| 5   | Invariantes de segurança                                      | 4 |
| 6   | O tenant como caso particular (desacoplamento)                | 5 |
| 7   | Caracterização algébrica                                      | 5 |
| 8   | Ciclo de vida: revogação e expiração                          | 7 |
| 9   | <i>Enforcement: pushdown no data layer</i>                    | 8 |
| A   | Plano de implementação                                        | 8 |

## 1 Introdução e motivação

A Patria opera uma plataforma multi-organização. Cada cliente (um *tenant*) controla os próprios cadastros, vendas, alunos, mensalidades; a Patria, como operadora, precisa *ver e operar* centralmente sobre os clientes; e dentro de cada cliente há níveis de acesso. Isso é *delegação*: a autoridade flui de quem a tem para quem a recebe.

Duas decisões de projeto. Primeiro, **o *tenant* não é especial**: em vez de um “pisso” fixo (`where:{tenantId}`) espalhado por centenas de *services*, o isolamento é apenas a condição  $c_t = (\text{tenantId} = t)$  embutida nas capacidades. Segundo — e é o ponto que distingue este modelo — toda concessão carrega uma **intensidade**  $\delta$  que limita a sua re-delegação. Sem isso, dar a um usuário acesso a dados de outro *tenant* contaminaria toda a cadeia abaixo dele; com  $\delta = 0$ , o acesso fica *só naquele usuário*.

Por fim, a estrutura de delegação **não é uma árvore**: um *tenant* pode conceder uma permissão *diretamente* a um usuário específico. O substrato é um *grafo acíclico dirigido* (DAG) de concessões.

## 2 Preliminares

**Definição 1** (Linhas e condições). Seja  $\mathcal{R}$  o universo de todas as linhas. Cada  $r \in \mathcal{R}$  tem atributos ( $r.\text{tenantId}$ ,  $r.\text{id}$ , ...). Uma *condição*  $c$  é um predicado  $c : \mathcal{R} \rightarrow \{\perp, \top\}$ , identificado ao seu *extent*  $\llbracket c \rrbracket = \{r \in \mathcal{R} : c(r)\} \subseteq \mathcal{R}$ . As condições formam a álgebra booleana  $(2^{\mathcal{R}}, \cap, \cup, \setminus, \emptyset, \mathcal{R})$  — as *MongoQuery* do CASL.

Fixamos *subjects*  $\mathcal{S}$  (**Customer**, **Student**, ...), *ações*  $\mathcal{A} = \{\text{create}, \text{read}, \text{update}, \text{delete}\}$  ( $\text{manage} = \bigvee \mathcal{A}$ ) e o curinga **all**.

**Definição 2** (Capacidade). Uma *capacidade* é uma tupla

$$\kappa = (a, s, c, \iota, \delta), \quad \delta \in \mathbb{N}_0 \cup \{\infty\},$$

com ação  $a$ , subject  $s$ , condição  $c$ , *flag* de inversão  $\iota$  ( $\iota=1$  é uma *proibição*, **cannot**) e *profundidade de propagação* (intensidade)  $\delta$ . Informalmente,  $\kappa$  autoriza (ou nega, se  $\iota=1$ )  $a$  sobre  $s$  nas linhas de  $\llbracket c \rrbracket$ , e admite ser re-delegada por mais  $\delta$  saltos.

## 3 Autorização de um principal

Seja  $P$  o conjunto de *principals* (operador, donos de *tenant*, perfis, usuários). Cada principal *detém* um conjunto de capacidades  $\text{Hold}(p)$  (Seção 4). Para um par fixo  $(s, a)$ , a autorização efetiva é a **união** dos grants menos as proibições:

$$E_p(s, a) = \left( \bigcup_{\substack{\kappa \in \text{Hold}(p) \\ \iota_\kappa=0, \kappa \vdash (s,a)}} \llbracket c_\kappa \rrbracket \right) \setminus \left( \bigcup_{\substack{\kappa \in \text{Hold}(p) \\ \iota_\kappa=1, \kappa \vdash (s,a)}} \llbracket c_\kappa \rrbracket \right),$$

onde  $\kappa \vdash (s, a)$  significa que  $\kappa$  casa o subject e a ação (com **manage**/**all** como curingas).

**Observação 3** (A intensidade não entra aqui).  $E_p$  não depende de  $\delta$ : a profundidade governa a *delegação* (Seção 4), não a avaliação. Quem detém uma capacidade a exerce plenamente, qualquer que seja o seu  $\delta$  restante. Isso é o que torna o *enforcement* barato (Seção 9).

**Observação 4** (União, não interseção). Compor grants com interseção seria um erro: “ler onde **org**=  $A$ ” e “ler onde **org**=  $B$ ” devem autorizar  $A \cup B$ . Só as proibições subtraem.

## 4 O grafo de delegação e a atenuação

**Definição 5** (Autoridades intrínsecas). Duas fontes de autoridade não derivam de ninguém:

- o **operador**  $\Omega$  detém  $(\text{manage}, \text{all}, \top, 0, \infty)$  — vê e faz tudo, com re-delegação irrestrita;
- o **dono do tenant**  $t$  detém  $(\text{manage}, \text{all}, (\text{tenantId}=t), 0, \infty)$  — controle total dos próprios recursos.

**Definição 6** (Grafo de delegação). Um *grafo de delegação* é um DAG cujos vértices são principais e cujas arestas  $u \xrightarrow{\kappa} w$  representam “ $u$  concede a capacidade  $\kappa$  a  $w$ ”. Note que  $w$  é *qualquer* principal — não há exigência de árvore nem de vizinhança: o dono de um *tenant* pode conceder *diretamente* a um usuário específico.

**Definição 7** (Concessão válida / atenuação). A aresta  $u \xrightarrow{\kappa} w$  é *válida* se existe  $\kappa_0 \in \text{Hold}(u)$  tal que

$$\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket, \quad \kappa \vdash (s, a) \Rightarrow \kappa_0 \vdash (s, a), \quad \iota_\kappa = \iota_{\kappa_0}, \quad \delta_{\kappa_0} \geq 1, \quad \delta_\kappa \leq \delta_{\kappa_0} - 1, \quad \tau_\kappa \leq \tau_{\kappa_0},$$

com a convenção  $\infty - 1 = \infty$ . Isto é,  $u$  só concede o que detém, *nunca ampliando* a condição, *sempre decrementando* a profundidade e *nunca estendendo* a validade  $\tau$  (expiração, Seção 8). Em particular  $\delta_{\kappa_0} = 0$  proíbe qualquer concessão.

**Definição 8** (Capacidades detidas).  $\text{Hold}(p)$  é o menor conjunto que contém as capacidades intrínsecas de  $p$  (se  $p$  é  $\Omega$  ou dono) e toda  $\kappa$  tal que existe uma aresta válida  $u \xrightarrow{\kappa} p$ .

#### 4.1 A intensidade $\delta$ : profundidade de propagação

Por construção (Definição 7), uma capacidade emitida com profundidade  $\delta$  admite cadeias de re-delegação de comprimento no máximo  $\delta$ :

$$\delta \xrightarrow{1 \text{ salto}} \delta - 1 \rightarrow \dots \rightarrow 0 \text{ (não re-delegável)}.$$

- $\delta = 0$ : o destinatário *usa* a capacidade, mas não a passa a ninguém. É o caso do “dar acesso sem permitir compartilhar”.
- $\delta = k$ : re-delegável por mais  $k$  saltos; cada salto decremena.
- $\delta = \infty$ : irrestrito (o comportamento legado, sem limite).

### 5 Invariantes de segurança

**Teorema 9** (Não-escalada por atenuação). *Para todo  $p$  e toda  $\kappa \in \text{Hold}(p)$ , existe uma autoridade intrínseca  $\kappa_0$  e uma cadeia de arestas válidas de sua origem até  $p$  tais que  $\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket$ . Logo*

$$E_p(s, a) \subseteq \bigcup_{\kappa_0 \text{ (raiz que alcança } p)} \llbracket c_{\kappa_0} \rrbracket.$$

*Nenhum principal acessa uma linha que nenhuma autoridade intrínseca em sua cadeia podia conceder.*

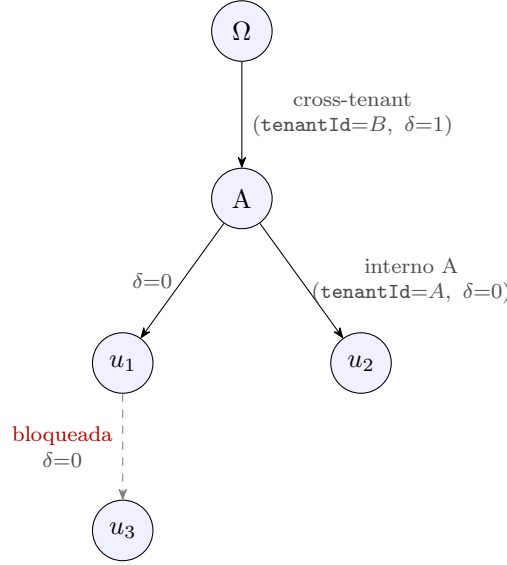


Figura 1: Grafo de delegação (DAG, não árvore).  $\Omega$  concede a  $A$  uma capacidade cross-tenant (`tenantId=B`) com  $\delta=1$ ;  $A$  pode re-delegá-la *uma* vez (a  $u_1$ , com  $\delta=0$ ), mas  $u_1$  *não* pode repassá-la (aresta bloqueada).  $A$  também concede algo interno diretamente a  $u_2$ . As arestas são explícitas —  $A$  alcança  $u_1, u_2$  diretamente, sem intermediários obrigatórios.

*Demonstração.* Indução no comprimento da cadeia de delegação. Caso base: capacidade intrínseca. Passo: a Definição 7 exige  $\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket$  a cada aresta; a inclusão é transitiva. A cota de  $E_p$  segue por monotonicidade da união.  $\square$

**Teorema 10** (Propagação limitada). *Uma capacidade emitida com profundidade  $\delta$  só pode aparecer (atenuada) ao fim de cadeias de re-delegação de comprimento  $\leq \delta$ . Em particular,  $\delta = 0 \Rightarrow$  não re-delegável: ela permanece exclusivamente no destinatário direto.*

*Demonstração.* Cada aresta válida decrementa  $\delta$  em pelo menos 1 e exige  $\delta_{\kappa_0} \geq 1$  (Definição 7); uma cadeia de comprimento  $\ell$  requer  $\delta \geq \ell$ . Para  $\delta = 0$  não há aresta de saída possível.  $\square$

**Corolário 11** (Compartilhamento controlado). *Conceder uma capacidade cross-tenant com  $\delta = 0$  garante que o destinatário a exerce mas ninguém abaixo dele a herda — exatamente “dar acesso sem propagar”. Com  $\delta = k$ , a difusão é confinada à vizinhança de re-delegação de raio  $k$ .*

**Observação 12** (Fail-closed = união vazia). Se nenhum grant detido casa  $(s, a)$ , então  $E_p(s, a) = \emptyset$ : acesso negado. É o *bottom* da álgebra, uniforme — sem piso de tenant especial.

## 6 O tenant como caso particular (desacoplamento)

**Corolário 13** (Isolamento de tenant). *Um usuário comum da organização  $t$  só detém capacidades atenuadas da capacidade intrínseca do dono, todas com  $\llbracket c \rrbracket \subseteq \llbracket \text{tenantId}=t \rrbracket$ . Logo  $E_p \subseteq \{r : r.\text{tenantId} = t\}$ : nenhuma linha de outro tenant aparece, sem regra especial. Um vazamento cross-tenant só ocorre por uma aresta de concessão explícita (auditável), cuja difusão é exatamente governada por  $\delta$ .*

O operador  $\Omega$ , detendo  $\top$ , enxerga todos os *tenants* — “eu, no topo, vejo tudo”. O *tenant* foi reduzido a uma condição.

## 7 Caracterização algébrica

Tudo o que precede é, na verdade, *um fluxo monótono sobre um reticulado booleano*. Tornar isso explícito entrega vários teoremas quase de graça.

**Definição 14** (O dioid das condições).  $(2^{\mathcal{R}}, \oplus, \otimes)$  com  $\oplus := \cup$  e  $\otimes := \cap$  é um *semianel idempotente comutativo* (dioid):  $\oplus$  é associativa, comutativa, idempotente ( $a \oplus a = a$ ), com neutro  $\emptyset$ ;  $\otimes$  idem, com neutro  $\mathcal{R}$ ; e  $\otimes$  distribui sobre  $\oplus$ . A *ordem canônica*  $a \preceq b \iff a \oplus b = b$  coincide com  $\subseteq$ . Com o complemento  $\neg$ , é a álgebra booleana completa  $(2^{\mathcal{R}}, \cup, \cap, \neg, \emptyset, \mathcal{R})$  — um *reticulado completo*.

**Observação 15** (Autorização é uma expressão do dioid). A Seção 3 é, literalmente, álgebra do semianel:

$$E_p(s, a) = \left( \bigoplus_g \llbracket c_g \rrbracket \right) \otimes \neg \left( \bigoplus_n \llbracket c_n \rrbracket \right),$$

soma dos grants vezes o complemento da soma das proibições. O *pushdown* (Seção 9) é apenas avaliar essa expressão.

**Definição 16** (Delegação é um operador monótono deflacionário). Cada aresta de delegação induz  $T : 2^{\mathcal{R}} \rightarrow 2^{\mathcal{R}}$  com

$$x \preceq y \Rightarrow T(x) \preceq T(y) \quad (\text{monótono}), \quad T(x) \preceq x \quad (\text{deflação: } T \preceq \text{id}).$$

A condição de atenuação  $\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket$  (Definição 7) é exatamente  $T \preceq \text{id}$ .

**Dois operadores duais (a distinção que importa).** A delegação tem *dois* lados, e é tentador — mas incorreto — fundir os dois. A **atenuação**  $T \preceq \text{id}$  (deflacionária, sobre os extents  $2^{\mathcal{R}}$ ) governa *o que*: o extent só encolhe. A **propagação**  $P$  governa *quem detém*: sobre o reticulado completo  $2^{\hat{P}}$  dos conjuntos de principals,

$$P(H) = H \cup \{ w : \exists u \in H, \text{ aresta válida } u \rightarrow w \}, \quad P \succeq \text{id} \text{ (inflacionária), monótona.}$$

**Os resultados de graça.**

- **Profundidade = iteração da propagação  $P$  (não de  $T$ ).** Re-delegar por  $\delta$  saltos é  $P^\delta$ . A intensidade  $\delta = k$  é o  $k$ -ésimo aproximante  $P^k(\{\text{orig}\})$  (reach em  $\leq k$  saltos);  $\delta = \infty$  é o **menor ponto fixo**  $\mu P = \bigsqcup_n P^n(\{\text{orig}\})$  (Kleene *de baixo*) — o fecho transitivo da delegação válida. É um  $\mu P$  *ascendente* sobre quem-detém, *não* um  $\nu T$  descendente sobre extents: as duas ideias não se identificam.
- **Não-escalada = monotonicidade de  $T$ .**  $p \mapsto E_p$  é um morfismo monótono do DAG de delegação (ordenado por derivação) para  $(2^{\mathcal{R}}, \preceq)$ . O Teorema 9 é a imagem da ordem.
- **Composição.** Operadores monótonos deflacionários são fechados sob  $\circ$ : a autoridade ao longo de um caminho é  $T_{e_1} \circ \dots \circ T_{e_k}$ , ainda monótona e deflacionária. Cadeias compõem sem sair da classe.
- **Fail-closed = zero.** Ausência de grant = soma vazia =  $\emptyset$  = neutro de  $\oplus$ . Negar é o zero do semianel (Teorema 10 e a Observação de fail-closed).
- **Auditoria por ordem parcial.**  $\text{Hold}(p)$  e a floresta de derivação são *posets* sob “derivado-de”; a revogação é o *fecho para baixo* (downset), uma operação monótona; expirar/revogar só decresce  $E_p$  — a FSM (Seção 8) é um fluxo monótono.

**Observação 17.** Os teoremas das seções anteriores deixam de ser fatos isolados: são corolários de “ $(2^{\mathcal{R}}, \cup, \cap)$  é um dioid e a delegação é um operador monótono  $\preceq \text{id}$ ”.

## 8 Ciclo de vida: revogação e expiração

Uma concessão não é eterna. Modelamos o seu *ciclo de vida* como uma máquina de estados finita (FSM), separada da álgebra de avaliação: a álgebra diz *o que* uma capacidade autoriza; a FSM diz *se* ela está em vigor.

**Definição 18** (Instância de concessão). Uma *instância* é uma capacidade  $\kappa$  acrescida de  $(\text{orig}, \text{par}, \tau, \sigma)$ : a *origem*  $\text{orig}$  (quem concedeu), o *pai*  $\text{par}$  (a instância da qual foi atenuada, ou  $\perp$  se intrínseca), a *expiração*  $\tau \in \mathbb{R}_{>0} \cup \{\infty\}$  e o *estado*  $\sigma \in \{\text{active}, \text{suspended}, \text{revoked}, \text{expired}\}$ . As arestas válidas (Definição 7) já exigem  $\tau_\kappa \leq \tau_{\kappa_0}$ : **um filho nunca sobrevive ao pai**.

**Definição 19** (Avaliação filtra o ciclo de vida). Em  $\text{Hold}(p)$  e em  $E_p(s, a)$  (Seção 3) contam *somente* as instâncias **vigentes** no instante  $t$ :  $\sigma = \text{active}$  e  $\tau > t$ . As demais não contribuem. Assim revogar/suspender/expirar uma concessão remove o seu extent da união sem tocar na álgebra.

**Teorema 20** (Revogação em cascata). *Revogar uma instância  $I$  revoga toda a sua descendência na floresta de derivação (as instâncias cuja cadeia de par passa por  $I$ ). Após a cascata, nenhuma instância vigente foi atenuada de  $I$ .*

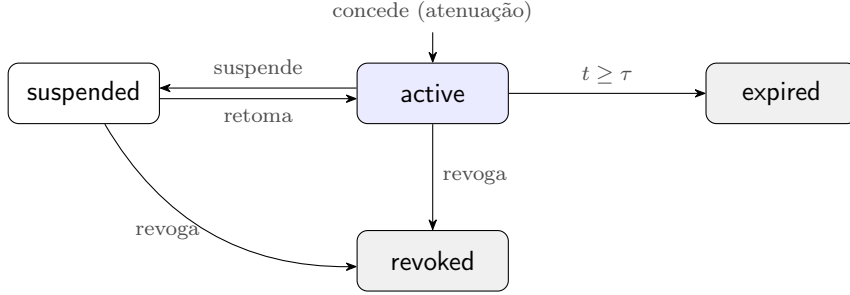


Figura 2: FSM do ciclo de vida de uma concessão. **revoked** e **expired** são terminais; só **active** contribui para a avaliação. *Guard* das transições destrutivas (suspende/revoga): o ator deve ter autoridade sobre a instância (ser a origem, um ancestral na cadeia, ou  $\Omega$ ).

*Demonstração.* Pela Definição 7, toda instância  $J$  atenuada (direta ou transitivamente) de  $I$  tem  $I$  na sua cadeia de pais. A cascata é a travessia dessa floresta a partir de  $I$ , marcando **revoked**. Como **revoked** é terminal e excluída da avaliação, nenhuma sobrevive.  $\square$

**Corolário 21** (Expiração cascadeia sozinha). *Como  $\tau_{\text{filho}} \leq \tau_{\text{pai}}$ , ao expirar o pai ( $t \geq \tau_{\text{pai}}$ ) todos os filhos já expiraram. A expiração não precisa de travessia — o invariante de atenuação a propaga.*

**Observação 22** (A FSM é o lugar certo; a avaliação não). A decisão de acesso permanece uma *computação pura* (a álgebra de §3–§6). A FSM modela apenas o que é genuinamente *stateful*: transições auditáveis e a cascata de revogação sobre o DAG. Os dois compõem sem se contaminar.

## 9 Enforcement: pushdown no data layer

A avaliação independe de  $\delta$  (Seção 4): basta empurrar  $E_p(s, a)$  para o **where**. Numa leitura de subject  $s$  pelo principal  $p$ ,

$$\text{retornado} = E_p(s, \text{read}) \cap \llbracket c_{\text{query}} \rrbracket.$$

Escritas validadas contra  $E_p(s, a)$ ; o conjunto de campos restringe colunas. Concretamente: uma *extensão do Prisma Client* (`$allOperations`) que injeta  $\bigcup(\text{grants}) \setminus \bigcup(\text{proibições})$  via **AND**.

**Observação 23** (Onde  $\delta$  é verificado). A intensidade é checada no **ato de delegar** — quando um principal cria/atribui uma capacidade a outro, valida-se a atenuação (Definição 7: condição  $\subseteq$ ,  $\delta' \leq \delta - 1$ ). A leitura de dados nunca consulta  $\delta$ . Isso mantém a extensão RLS simples e a auditoria concentrada no momento da concessão.

**Observação 24** (Dois modos e o piso nativo). `shadow` computa e *registra* onde negaria sem alterar o resultado; `enforce` aplica. `$queryRaw` escapa da extensão de aplicação; por isso um teste-*guardrail* barra *raw* fora de uma *allowlist* escopada, e a Fase B adiciona *Row-Level Security* nativa do PostgreSQL como piso duro.

## A Plano de implementação

**Escopo (backbone).** A avaliação (*pushdown* da união de capacidades detidas) já está implementada e testada (`condition-resolver.ts`, `prisma-rls.extension.ts`): ela *independe* de  $\delta$ , logo vale desde já. A camada de *delegação* com  $\delta$  (grafo, atenuação) entra com o multi-org; o campo e o primitivo de atenuação são preparados agora.

**A1. Capacidade carrega  $\delta$ .** Adicionar `propagationDepth Int?` a `AccessRule` (`null`=  $\infty$ ; 0 = não re-delegável) e a `RuleInput`. Migração nullable, retro-compatível (regras atuais =  $\infty$ ).

**A2. Primitivo de atenuação (testado).** `access/delegation.ts`: `attenuatedDepth(holder, proposed)` ( $\infty-1 = \infty$ ; exige `holder`  $\geq 1$ ; `proposed`  $\leq holder-1$ ) e `canDelegate`. Verificado no serviço de concessão (criar regra / atribuir perfil), *não* na leitura.

**A3. Avaliação (já feita).** `condition-resolver.ts`:  $E_p$  = união dos grants \ proibições, sob o escopo herdado. `prisma-rls.extension.ts`: injeção no `where`, OR nos grants, fail-closed, guarda de *create* cross-escopo, modos `shadow/enforce`. Mapa *Subject*  $\leftrightarrow$  *model*; *registry* via DMMF.

**A4. Wiring.** `TenantContext` carrega regras + escopo + `rlsMode`; `PrismaService` vira *Proxy* por-request com client estendido; `PoliciesGuard` reusa as regras do contexto.

**A5. Testes (por funcionalidade — exigência central).** `condition-resolver.spec`, `prisma-rls.extension.spec` (por *subject/model*), `delegation.spec` (atenuação/ $\delta$ ), `ability.contract.spec` (matriz dos perfis), *guardrail* de *raw*. Regra de projeto: nenhuma capacidade/perfil novo sem teste.

**A6. Gate na CI.** `job test (npm ci + prisma generate + npm test)`; `deploy` com `needs: test`.

**A7. Rollout.** `RLS_MODE=shadow`  $\rightarrow$  `enforce`. Fase B: RLS nativa do PostgreSQL como piso duro.

**A8. Ciclo de vida (FSM) — Seção 8.** `AccessRule` ganha `state` (active/suspended/ revoked/expired), `expiresAt` `DateTime?` (null=  $\infty$ ) e `parentId` (auto-relação para a cascata). A avaliação passa a filtrar `state='active'` AND (`expiresAt` IS NULL OR `expiresAt` > `now()`) — some uma cláusula na carga das regras, a álgebra não muda. Revogar = `UPDATE state='revoked'` em `I` e em toda a sua descendência por `parentId` (CTE recursiva). Atenuação de  $\tau$  no *grant-check* (espelho de `attenuatedDepth`). Transições auditadas; *guard* = autoridade sobre a instância.